

Contents

OpenAI Codex CLI – Comprehensive Architecture Document	2
1. Overview	2
2. Architecture	3
2.1 Original Stack (April 2025)	3
2.2 Rust Rewrite (codex-rs)	3
2.3 App Server Architecture	3
2.4 Agent Loop	3
3. Core Features	4
3.1 Interactive Terminal UI (TUI)	4
3.2 Non-Interactive Automation (Exec Mode)	4
3.3 File Editing via <code>apply_patch</code>	4
3.4 Code Generation and Multi-File Changes	4
3.5 Web Search	4
3.6 Cloud Task Delegation	5
3.7 Subagents	5
4. Models	5
4.1 Model Evolution	5
4.2 Context Windows	5
4.3 Pricing	6
4.4 Reasoning Levels	6
5. Tool System	6
5.1 Built-in (Default Solver) Tools	6
5.2 How Tools Work	6
5.3 Tool Extensibility via MCP	6
6. MCP Support	6
6.1 Supported Transports	7
6.2 Configuration	7
6.3 Compatible MCP Servers	7
6.4 MCP Limitations	7
7. Skills / Extensions / Custom Instructions	7
7.1 AGENTS.md	7
7.2 Custom Prompts (Slash Command Prompts)	8
7.3 Personalities	8
8. Agent Capabilities & Approval Modes	8
8.1 Approval Modes	8
8.2 Convenience Presets	9
8.3 Granular Approval Policy	9
8.4 Managed Constraints	9
9. Sandbox System	9
9.1 Sandbox Modes	9
9.2 OS-Level Implementation	9
9.3 Workspace-Write Configuration	10
9.4 Automatic Mode Detection	10
10. IDE Integration	10
10.1 Supported Editors	10
10.2 Platform Support	10
10.3 IDE Features	10
10.4 Architecture	10
11. Session Management	10
11.1 Local Transcript Storage	10
11.2 Resume Command	11
11.3 Cross-Surface Continuity	11
12. Configuration	11
12.1 Configuration Layers (Highest Precedence First)	11
12.2 Key Configuration Sections	11

12.3 Project-Scoped Configuration	12
12.4 TOML Format Note	12
13. Permissions & Security	12
13.1 Two-Layer Security Model	12
13.2 Principle of Least Privilege	12
13.3 GitHub Action Security	12
13.4 Managed Security	12
14. Context Management	12
14.1 Automatic Context Compaction	12
14.2 Manual Compaction	13
14.3 Prompt Caching	13
14.4 File Context	13
15. Unique Features (What Codex CLI Does Differently)	13
15.1 Open Source (Apache 2.0)	13
15.2 Multi-Surface Unified Architecture	13
15.3 Rust-Native with Zero Dependencies	13
15.4 GitHub Deep Integration	13
15.5 Cloud + Local Hybrid	13
15.6 AGENTS.md as Open Standard	13
15.7 Adjustable Reasoning Effort	13
15.8 Built-in Web Search with Cache Mode	13
15.9 Codex SDK	14
16. Limitations	14
16.1 Known Gaps	14
16.2 Comparison Gaps vs Claude Code	14
16.3 Cost Considerations	14
17. Version History	14
Major Milestones	14
Architecture Evolution	15
18. Command Reference (Quick Reference)	15
CLI Commands	15
Slash Commands (In-Session)	16
19. Installation	16
Sources	16

OpenAI Codex CLI – Comprehensive Architecture Document

Last updated: 2026-03-25 Current version: ~0.117.x (Rust-native, alpha track) Repository: <https://github.com/openai/codex> License: Apache 2.0

1. Overview

OpenAI Codex CLI is an open-source, terminal-based AI coding agent built by OpenAI. It launched on **April 16, 2025** alongside the release of the o3, o4-mini, and GPT-4.1 model families. The tool runs locally on a developer’s machine, reads and edits files, executes shell commands, and leverages OpenAI’s reasoning models to assist with software engineering tasks.

Codex CLI is one of four surfaces in the broader **Codex ecosystem**:

Surface	Description
Codex Web	Cloud-based agent at chatgpt.com/codex
Codex CLI	Open-source terminal agent (this document)
Codex IDE Extension	VS Code / Cursor / Windsurf extension
Codex macOS App	Native desktop app launched February 2, 2026

All four surfaces are powered by the same **Codex App Server** – a bidirectional JSON-RPC protocol that decouples the agent’s core logic from the client UI.

At launch, Codex CLI was bundled with a \$1 million API grants program (\$25,000 blocks) to support early adopters.

2. Architecture

2.1 Original Stack (April 2025)

The original Codex CLI was built with: - **TypeScript / Node.js** (Node v22+ required) - **React** (Ink) for the terminal UI (TUI) - OpenAI **Responses API** for model inference (not Chat Completions) - OS-native sandbox primitives for security enforcement

2.2 Rust Rewrite (codex-rs)

In June 2025, OpenAI announced the rewrite of Codex CLI from TypeScript to **Rust**, designated **codex-rs** in the repository. Motivations:

- **Zero-dependency install** – eliminates the Node.js 22+ requirement
- **Native security bindings** – direct Seatbelt (macOS), Bubblewrap/Landlock (Linux), Restricted Tokens (Windows) integration
- **Performance** – faster startup, lower memory, higher throughput
- **Wire protocol** – a stable protocol allowing extensions in any language (TypeScript, Python, etc.)

The Rust version is now the maintained default. The first official Rust release was **rust-v0.2.0**. As of March 2026, the version is in the **0.117.x** alpha range.

2.3 App Server Architecture

All Codex experiences (CLI, IDE, web, macOS app) communicate through the **Codex App Server**:

[Client (CLI/IDE/Web)] <--JSON-RPC over JSONL/stdio--> [App Server] <--Responses API--> [OpenAI Models]

Key design decisions: - **Bidirectional JSON-RPC** streamed as JSONL over stdio - **Backward compatible** – older clients can safely communicate with newer server versions - **Item-based primitives** – each piece of input/output has a lifecycle: **started** -> **delta** (streaming) -> **completed** - **Server-initiated requests** – when the agent needs approval, the server sends a request to the client and pauses until it receives **allow** or **deny**

OpenAI explicitly **rejected MCP** as the internal protocol between client and server. When building the VS Code extension, the team experimented with exposing Codex as an MCP server but found that “maintaining MCP semantics in a way that made sense for VS Code proved difficult.” The richer session semantics (streaming diffs, approval flows, thread persistence) did not map cleanly onto MCP’s tool-oriented model.

2.4 Agent Loop

The core agent loop follows a classical agentic pattern with production optimizations:

```
User Input
|
v
Build Prompt (instructions + tools + context + user message)
|
v
Call Responses API (streaming via SSE)
|
v
Response = Final Answer? ---YES---> Display to User
|
NO (tool call requested)
|
```

```

      v
Execute Tool (shell, apply_patch, read_file, etc.)
      |
      v
Append Tool Output to Prompt
      |
      v
Re-query Responses API
      |
(loop continues)

```

Performance optimization – Prompt Caching: The agent loop is inherently quadratic (growing JSON payloads). OpenAI's prompt caching achieves **linear-time sampling** by reusing intermediate computations when the new prompt shares an exact prefix match with a previous call. Codex maximizes cache hits by placing static content (instructions, tool definitions, sandbox config) at the prompt's beginning.

3. Core Features

3.1 Interactive Terminal UI (TUI)

Codex launches into a full-screen terminal UI where you can: - Send prompts, code snippets, or screenshots into the composer - Watch Codex explain its plan before making changes - Approve or reject steps inline - Use @ to fuzzy-search and reference files in the workspace - Use / to access slash commands

3.2 Non-Interactive Automation (Exec Mode)

```
codex exec "Update all npm dependencies to their latest compatible versions and run tests"
```

Runs Codex non-interactively, piping the final plan and results to stdout. Designed for scripting, CI/CD pipelines, and automated code maintenance.

3.3 File Editing via apply_patch

Codex uses a custom **apply_patch** tool for file modifications – a stripped-down, file-oriented diff format:

```

*** Begin Patch
*** Update File: src/utils.ts
@@ -10,3 +10,4 @@
    function helper() {
        return true;
+   // Added logging
    }
*** End Patch

```

Operations supported: - ***** Add File:** – create a new file - ***** Delete File:** – remove an existing file - ***** Update File:** – patch in place (with optional ***** Move to:** for renames)

File references are always **relative, never absolute**. The CLI intercepts **apply_patch** commands, processes them internally, and displays colorized diffs (red for deletions, green for additions).

3.4 Code Generation and Multi-File Changes

Codex can: - Read and analyze entire repositories - Generate new files and boilerplate - Make coordinated changes across multiple files - Run tests, linters, and build commands - Execute git operations

3.5 Web Search

Codex ships with a **first-party web search tool**, enabled by default for local tasks:

Mode	Behavior
<code>cached</code> (default)	Returns results from OpenAI-maintained pre-indexed cache. Reduces prompt injection risk.
<code>live</code>	Fetches the most recent data from the web in real-time. Default when using <code>--yolo</code> .

Configuration: `web_search = "cached"` or `web_search = "live"` in `config.toml`.

3.6 Cloud Task Delegation

The `codex cloud` command lets you: - Triage and launch cloud tasks without leaving the terminal - Browse active or finished tasks - Apply cloud-generated changes to your local project - Parallelize large refactors across multiple cloud instances

3.7 Subagents

Codex can spawn **subagents** for parallel task execution: - Only spawned when you explicitly ask - Useful for codebase exploration, multi-step feature plans - `spawn_agents_on_csv` for batch processing similar tasks (one worker per row) - Configuration: `agents.max_depth = 1` (default) prevents deeper nesting - Each subagent does its own model and tool work (consumes more tokens than single-agent runs)

4. Models

4.1 Model Evolution

Model	Release Period	Notes
o3	April 2025	Launch model, 200K context, leading reasoning
o4-mini	April 2025	Launch model, 200K context, cost-efficient reasoning
GPT-4.1	April 2025	1M token context, instruction following, tool calling
codex-mini-latest	Mid-2025	Fine-tuned o4-mini for Codex CLI
GPT-5-Codex	Late 2025	First GPT-5 series for Codex
GPT-5-Codex-Mini	Nov 2025	Smaller, ~4x more usage per subscription
GPT-5.1-Codex-Max	Late 2025	Frontier model, faster, more token-efficient
GPT-5.2-Codex	Early 2026	Context compaction, repo-scale reasoning, Windows improvements
GPT-5.3-Codex	2026	Leads Terminal-Bench 2.0 at 77.3%, 240+ tokens/sec
GPT-5.4	Current recommended	Industry-leading coding, native computer use

4.2 Context Windows

Model Family	Context Window
o3 / o4-mini	200K tokens
GPT-4.1	1M tokens
GPT-5.x Codex variants	Varies (typically 200K-1M)

4.3 Pricing

API (pay-as-you-go): - `codex-mini-latest`: \$1.50 / 1M input tokens, \$6.00 / 1M output tokens - `GPT-5`: \$1.25 / 1M input tokens, \$10.00 / 1M output tokens

ChatGPT Subscription (CLI + Web): - Plus (\$20/month): 30-150 messages per 5 hours - Pro (\$200/month): 300-1,500 messages per 5 hours

4.4 Reasoning Levels

Codex supports adjustable reasoning effort: `minimal`, `low`, `medium`, `high`. Higher effort helps on complex tasks but consumes more tokens and time. Use `/model` slash command or `--model` flag to switch.

5. Tool System

5.1 Built-in (Default Solver) Tools

Tool	Purpose
<code>shell</code>	Execute arbitrary shell commands in the sandbox
<code>apply_patch</code>	Apply file edits in the custom patch format
<code>read_file</code>	Read file contents (preferred over <code>cat</code>)
<code>list_dir</code>	List directory contents
<code>glob_file_search</code>	Search for files by glob pattern
<code>git</code>	All git operations
<code>rg</code> (ripgrep)	Code search
<code>todo_write</code> / <code>update_plan</code>	Track and update task plans
<code>web_search</code>	Search the web (cached or live)

5.2 How Tools Work

1. The model generates a function call (e.g., `type: "function_call", name: "shell", arguments: {...}`)
2. The CLI intercepts the call, checks approval policy
3. If approved, executes the tool within the sandbox
4. Tool output is appended to the conversation
5. The model receives the output and continues reasoning

For `apply_patch` specifically, the model generates a shell function call containing the patch content, and the CLI intercepts it, parses the patch format, displays a colorized diff, and waits for user approval.

5.3 Tool Extensibility via MCP

Additional tools can be connected via Model Context Protocol servers (see Section 6).

6. MCP Support

Codex CLI **does** support MCP (Model Context Protocol) for connecting to external tools and data sources. It supports both the CLI and the IDE extension.

6.1 Supported Transports

Transport	Description
STDIO	Local process, started by a command
Streamable HTTP	Remote server accessed at an address

6.2 Configuration

MCP servers are configured in `config.toml`:

~/.codex/config.toml (global) or .codex/config.toml (project-scoped, trusted only)

```
[[mcp_servers]]
name = "openai-docs"
transport = "stdio"
command = "npx"
args = ["-y", "@openai/codex-mcp-docs"]

[[mcp_servers]]
name = "my-remote-server"
transport = "streamable-http"
url = "https://my-mcp-server.example.com/mcp"
```

Management commands:

```
codex mcp add <name> <command>      # Add a new STDIO server
codex mcp list                       # List configured servers
codex mcp remove <name>              # Remove a server
```

Codex launches MCP servers automatically when a session starts and exposes their tools alongside built-ins.

6.3 Compatible MCP Servers

Server	Purpose
OpenAI Docs MCP	Search and read OpenAI developer docs
Context7	Up-to-date developer documentation
Figma (Local/Remote)	Access Figma designs
Playwright	Browser control and inspection
Chrome DevTools	Chrome control and inspection
Sentry	Access Sentry logs
GitHub	PR, issue management beyond git

6.4 MCP Limitations

- Known issue: CLI only checks `resources/list` for MCP availability, though it is optional in the MCP spec
- OAuth resource indicator handling has compatibility gaps

7. Skills / Extensions / Custom Instructions

7.1 AGENTS.md

Codex reads `AGENTS.md` files **before doing any work**. The lookup order per directory:

1. `AGENTS.override.md`
2. `AGENTS.md`
3. `TEAM_GUIDE.md`

4. .agents.md

Files are checked in each directory from root to current working directory. This is the same open standard used by Cursor, Aider, and other tools – if your team already has **AGENTS.md**, Codex inherits it.

Example **AGENTS.md**:

```
# Project Guidelines

## Code Style
- Use TypeScript strict mode
- Prefer functional components with hooks
- All new files must include JSDoc comments

## Testing
- Write unit tests for all utility functions
- Use vitest for testing

## Architecture
- Follow feature-based folder structure
- Keep components under 200 lines
```

7.2 Custom Prompts (Slash Command Prompts)

Custom prompts turn Markdown files into reusable slash commands:

```
~/.codex/prompts/draftpr.md
~/.codex/prompts/review.md
```

Invocation: `/prompts:draftpr` in the CLI composer.

Custom prompts: - Require explicit invocation - Live in `~/.codex/` (not shared through repository) - Work in both CLI and IDE extension

7.3 Personalities

Codex ships with configurable communication styles:

Personality	Behavior
friendly	Warm, checks in often, explains without ego, supportive pairing
pragmatic	Direct, focused on efficiency
none	Disables personality instructions

Configuration: `personality = "friendly"` in `config.toml`, or `/personality` slash command mid-session.

8. Agent Capabilities & Approval Modes

8.1 Approval Modes

Approval modes control when Codex must pause and ask for confirmation:

Mode	Behavior
untrusted	Maximum friction; asks before most actions
on-request	Moderate friction; asks for sensitive operations
never	No approval prompts (requires explicit opt-in)
granular	Per-category control with allow/auto-reject rules

8.2 Convenience Presets

Flag	Effect
<code>--full-auto</code>	Sets <code>approval_policy = on-request + sandbox = workspace-write</code> . Low-friction local automation.
<code>--yolo /</code> <code>--dangerously-bypass-approvals-and-sandbox</code>	Bypasses all approval prompts and sandboxing. Use only in isolated runners.
<code>--search</code>	Enables live web search (<code>web_search = "live"</code>)

8.3 Granular Approval Policy

```
[approval_policy.granular]
sandbox_approval = "on-request"

[[approval_policy.granular.rules]]
category = "network_access"
action = "deny"

[[approval_policy.granular.rules]]
category = "file_write_outside_workspace"
action = "ask"
```

8.4 Managed Constraints

Organizations can enforce constraints via `requirements.toml`: - Disallow `approval_policy = "never"` - Disallow `sandbox_mode = "danger-full-access"` - Enforce minimum sandbox levels

9. Sandbox System

9.1 Sandbox Modes

Mode	File Access	Network	Use Case
<code>read-only</code>	Read only	Blocked	Default for non-VCS folders; consultative mode
<code>workspace-write</code>	Read all, write within workspace	Blocked by default	Default for VCS folders; standard development
<code>danger-full-access</code>	Full read/write	Full access	Only for trusted repos/tasks; use sparingly

9.2 OS-Level Implementation

Codex translates high-level sandbox requirements into OS-native primitives:

Platform	Mechanism	Notes
macOS	Seatbelt (<code>sandbox-exec</code>)	Dynamically generates SBPL (Sandbox Profile Language) scripts based on requested permissions
Linux	Bubblewrap / Landlock	User namespace unsharing for consistent isolation; <code>/usr</code> , <code>/bin</code> , <code>/lib</code> mounted read-only
Windows	Restricted Tokens	Experimental support

9.3 Workspace-Write Configuration

```
[sandbox_workspace_write]
# Additional directories Codex can write to
writable_roots = ["/tmp/build-output", "/home/user/shared-libs"]

# Exclude temp directories from writable roots
exclude_tmp = false
exclude_tmpdir = false

# Allow network access within workspace-write mode
allow_network = false
```

9.4 Automatic Mode Detection

On launch, Codex detects whether the folder is version-controlled: - **VCS folder** -> recommends **Auto** mode (workspace-write + on-request approvals) - **Non-VCS folder** -> recommends **read-only** mode

10. IDE Integration

10.1 Supported Editors

- **VS Code** (primary)
- **Cursor**
- **Windsurf**
- Other VS Code-compatible editors

10.2 Platform Support

Platform	Status
macOS	Full support
Linux	Full support
Windows	Experimental (WSL recommended)

10.3 IDE Features

- **Chat and Code Context:** Uses open files and selected code for context; @ references any file
- **Image Input:** Drag and drop images (hold Shift) into the prompt composer
- **Model Selection:** Switch models and reasoning levels from the extension
- **Cloud Task Delegation:** Launch cloud tasks from the IDE, preview changes, continue locally
- **Command Palette:** All Codex commands available as keyboard shortcuts
- **Shared Configuration:** Same config.toml and AGENTS.md as the CLI
- **MCP Support:** Same MCP server configuration as the CLI
- **Web Search:** Enabled by default for local tasks

10.4 Architecture

The IDE extension uses the **Codex App Server** harness to drive the same agent loop from an IDE UI without reimplementing it. It supports rich interaction patterns: workspace exploration, streaming progress, emitting diffs, approval flows, and thread persistence.

11. Session Management

11.1 Local Transcript Storage

Codex stores transcripts locally so you can pick up where you left off instead of repeating context.

11.2 Resume Command

`codex resume`

Reopens an earlier thread with the same repository state and instructions. When you relaunch, you see past projects and can select one to continue.

11.3 Cross-Surface Continuity

Sessions started in the CLI can be continued in the IDE extension or web app, and vice versa, as long as they share the same project context. The `codex cloud` command bridges local and cloud sessions.

12. Configuration

12.1 Configuration Layers (Highest Precedence First)

1. Command-line flags (`--model`, `--sandbox`, etc.)
2. Environment variables
3. Project config files (`.codex/config.toml`, from project root down to CWD)
4. User config (`~/.codex/config.toml`)
5. Managed config (`requirements.toml` for organizations)
6. Defaults

12.2 Key Configuration Sections

~/.codex/config.toml -- Example configuration

Model settings

`model = "gpt-5.4"`

`personality = "pragmatic"`

Approval policy

`approval_policy = "on-request"`

Or granular:

[approval_policy.granular]

`sandbox_approval = "on-request"`

Sandbox settings

`sandbox_mode = "workspace-write"`

`[sandbox_workspace_write]`

`writable_roots = []`

`allow_network = false`

Context management

`model_context_window = 200000`

`model_auto_compact_token_limit = 150000`

Web search

`web_search = "cached"`

Subagents

`[agents]`

`max_depth = 1`

MCP servers

`[[mcp_servers]]`

`name = "openai-docs"`

```
transport = "stdio"
command = "npx"
args = ["-y", "@openai/codex-mcp-docs"]
```

12.3 Project-Scoped Configuration

Place `.codex/config.toml` in your repository root for project-specific settings. Project-scoped MCP servers only work in **trusted projects**.

12.4 TOML Format Note

Root keys must appear **before** tables in TOML. Place `model = "..."` and `approval_policy = "..."` before `[[sandbox_workspace_write]]` or `[[mcp_servers]]`.

13. Permissions & Security

13.1 Two-Layer Security Model

Layer	Controls
Sandbox mode	What Codex can technically do (filesystem writes, network access)
Approval policy	When Codex must ask before executing

13.2 Principle of Least Privilege

Defaults are conservative: - No network access - Write permissions limited to the active workspace - Approval required for edits outside workspace or network access

13.3 GitHub Action Security

The Codex GitHub Action (`openai/codex-action@v1`) includes a critical safety measure: **it drops sudo permissions** so Codex cannot access its own OpenAI API key. This prevents credential exfiltration, especially important for public repositories.

13.4 Managed Security

Organizations can enforce constraints via `requirements.toml`: - Prevent users from disabling sandboxing - Require minimum approval levels - Lock down configuration options

14. Context Management

14.1 Automatic Context Compaction

When the conversation approaches the context window limit, Codex automatically **compacts** the conversation:

- 1. Detects token count exceeding threshold
- 2. Calls a special Responses API endpoint
- 3. Receives a smaller, representative summary
- 4. Replaces previous input with the compacted version

Configuration:

```
model_context_window = 200000 # Available context tokens
model_auto_compact_token_limit = 150000 # Trigger threshold for compaction
```

14.2 Manual Compaction

The `/compact` slash command manually triggers compaction, which queries the Responses API with the existing conversation plus summarization instructions.

14.3 Prompt Caching

Static content (instructions, tool definitions, sandbox config) is placed at the beginning of prompts to maximize **prompt cache hits**. Variable user messages are appended at the end. This allows the API to reuse intermediate computations, achieving linear-time sampling despite quadratic payload growth.

14.4 File Context

- Use `@` in the composer for fuzzy file search
 - Use `/mention <path>` to add files to conversation context
 - Codex reads `AGENTS.md` files automatically for project context
-

15. Unique Features (What Codex CLI Does Differently)

15.1 Open Source (Apache 2.0)

Unlike Claude Code (closed source), Codex CLI is fully open source under Apache 2.0. Developers can fork, customize, and contribute.

15.2 Multi-Surface Unified Architecture

The same App Server protocol powers CLI, IDE extension, web app, and macOS desktop app. No other coding agent has this level of surface unification.

15.3 Rust-Native with Zero Dependencies

The Rust rewrite provides a single binary with no runtime dependencies (no Node.js, no Python). Download and run.

15.4 GitHub Deep Integration

- `@codex review` in PRs triggers automated code review
- **Codex GitHub Action** for CI/CD automation
- Inline comments, auto-fix suggestions, PR creation
- Works with Slack (`@Codex` in channels) and Linear (mention in issues)

15.5 Cloud + Local Hybrid

Codex can parallelize large refactors across multiple cloud instances while maintaining a local CLI workflow. The `codex cloud` command bridges the two.

15.6 AGENTS.md as Open Standard

By adopting `AGENTS.md` (shared with Cursor, Aider, and others), Codex inherits existing team configurations without proprietary lock-in.

15.7 Adjustable Reasoning Effort

Unlike most tools with fixed reasoning, Codex offers `minimal`, `low`, `medium`, `high` reasoning levels per request, letting developers trade speed for accuracy.

15.8 Built-in Web Search with Cache Mode

The cached web search mode reduces prompt injection risk by serving from an OpenAI-maintained index rather than live web pages.

15.9 Codex SDK

The TypeScript SDK (@openai/codex-sdk) wraps the CLI, spawning it as a subprocess and exchanging JSONL events over stdin/stdout. This enables embedding Codex in custom applications:

```
import { Codex } from "@openai/codex-sdk";

const codex = new Codex({
  env: { PATH: "/usr/local/bin" }, // Sandboxed environment
});
```

16. Limitations

16.1 Known Gaps

- **Windows support is experimental** – best experience requires WSL
- **Binary file operations** in PRs limited to delete/rename only
- **MCP OAuth handling** has compatibility gaps with providers requiring OAuth resource indicators
- **MCP availability detection** only checks `resources/list`, which is optional in the MCP spec
- **Sandbox regressions** on Linux/WSL with Bubblewrap (missing `writable_roots` can fail command startup)
- **apply_patch** breakage occurred in v0.116.0 (`bwrap: Unknown option --argv0`)
- **VS Code extension** on macOS emits error floods during activation (not present in terminal CLI)
- **Git Bash compatibility** issues even with sandbox disabled
- **No HTTP MCP endpoint support** natively (STDIO only; Streamable HTTP is newer)

16.2 Comparison Gaps vs Claude Code

Area	Codex CLI	Claude Code
SWE-bench Verified	69.1%	72.7%
Deterministic refactors	Variable across runs	More consistent
Clarifying questions	Tends to execute immediately	Asks before acting
Closed-source security	Open source (inspectable)	Closed source (controlled)
Context management	Automatic compaction	Agentic search + context file

16.3 Cost Considerations

For heavy API usage, costs can reach \$20-\$50/day or \$100+ /day for complex projects. The subscription model (Plus/Pro) offers more predictable pricing but with message limits.

17. Version History

Major Milestones

Date	Event
April 16, 2025	Codex CLI v0.1 launched (TypeScript/Node.js). Open source under Apache 2.0. Launch models: o3, o4-mini, GPT-4.1.
Mid-2025	codex-mini-latest model released (fine-tuned o4-mini for Codex).
June 2025	Rust rewrite announced (codex-rs). First Rust release: rust-v0.2.0.
Late 2025	GPT-5-Codex and GPT-5.1-Codex-Max models added.

Date	Event
November 2025	GPT-5-Codex-Mini launched (~4x more usage per subscription).
Early 2026	GPT-5.2-Codex released with context compaction improvements.
February 2, 2026	Codex macOS desktop app launched. Codex goes generally available.
February 2026	Codex App Server architecture published (JSON-RPC protocol). IDE extension launched for VS Code/Cursor/Windsurf.
March 2026	Current version ~0.117.x (Rust alpha). Slack and Linear integrations. GPT-5.4 recommended model.

Architecture Evolution

April 2025:	TypeScript + React (Ink) + Node.js 22+
June 2025:	Rust rewrite announced (codex-rs)
Late 2025:	Parallel development (TS for stability, Rust for features)
Early 2026:	Rust becomes default. App Server protocol published.
March 2026:	0.117.x alpha. Zero-dependency native binary.

18. Command Reference (Quick Reference)

CLI Commands

```
# Start interactive session
codex

# Non-interactive execution
codex exec "your prompt here"

# Resume a previous session
codex resume

# Cloud task management
codex cloud

# MCP server management
codex mcp add <name> <command>
codex mcp list
codex mcp remove <name>

# Full-auto mode (workspace-write + on-request approvals)
codex --full-auto "refactor auth module"

# Specific model
codex --model gpt-5.4

# Live web search
codex --search "find the latest React 19 API changes"
```

```
# Bypass everything (dangerous, CI only)
codex --yolo "run full test suite and fix failures"
```

Slash Commands (In-Session)

Command	Purpose
/model	Switch model or reasoning level
/fast	Switch to faster model
/permissions	Change approval preset
/personality	Change communication style
/diff	View git diff of changes
/mention <path>	Add file to conversation context
/compact	Manually trigger context compaction
/agent	Manage subagents
/status	View session status
/prompts:<name>	Invoke a custom prompt

19. Installation

```
# npm (requires Node.js 18+ for SDK, not needed for native binary)
npm install -g @openai/codex

# Homebrew
brew install codex

# Direct binary download (recommended -- zero dependencies)
# Download from https://github.com/openai/codex/releases
# Archives contain a single binary with the platform baked into the name
```

First run prompts for authentication via ChatGPT account or API key.

Sources

- [OpenAI Codex CLI Official Docs](#)
- [OpenAI Codex CLI Features](#)
- [GitHub: openai/codex](#)
- [Codex CLI Command Reference](#)
- [Codex Configuration Reference](#)
- [Codex Sample Configuration](#)
- [Codex Sandboxing](#)
- [Codex Agent Approvals & Security](#)
- [Codex MCP Support](#)
- [AGENTS.md Guide](#)
- [Codex Custom Prompts](#)
- [Codex Subagents](#)
- [Codex IDE Extension](#)
- [Codex Slash Commands](#)
- [Codex Models](#)
- [Codex Pricing](#)
- [Codex Changelog](#)
- [Codex GitHub Releases](#)
- [Codex GitHub Action](#)
- [Codex App Server Architecture](#)

- [Unrolling the Codex Agent Loop \(OpenAI Blog\)](#)
- [Unlocking the Codex Harness: App Server \(OpenAI Blog\)](#)
- [Introducing Codex \(OpenAI Blog\)](#)
- [Codex CLI Rust Rewrite \(InfoQ\)](#)
- [Codex CLI Going Native \(GitHub Discussion\)](#)
- [Codex vs Claude Code \(Builder.io\)](#)
- [Codex vs Claude Code \(Northflank\)](#)
- [Codex SDK \(npm\)](#)
- [Codex SDK Docs](#)
- [How Codex Works Behind the Scenes \(PromptLayer\)](#)